

College Event Management Platform

Simple Project Overview

1. What are we building?

We are building a **reusable college event management platform** where students can discover and register for events, organizers can manage participants and attendance, and a **Main Admin** can control the entire platform.

The important point is that the website will **not be created for one fixed event**.

When the system is first deployed, it can be mostly empty. The Main Admin will add the required:

- Domains
- Categories
- Events
- Sessions
- Fees
- Organizers
- Registration settings

So the same website can be reused for future events.

2. Who will use the platform?

There are three main users.

Main Admin

The Main Admin controls everything.

They can:

- Add domains
- Add categories
- Create events

- Add sessions
- Set event fees
- Set event capacity
- Open/close registrations
- Add organizers
- Assign organizers to events
- View students
- View registrations
- View payments
- View attendance
- View reports

Student

Students can:

- Create an account
- Complete their profile
- Pay the initial platform registration fee
- Browse events
- Filter/search events
- View event details
- Mark events as interested
- Register for events
- Pay event fees
- View their registrations
- Get their personal QR code
- Use the QR code for attendance

Organizer

Organizers manage the events assigned to them.

They can:

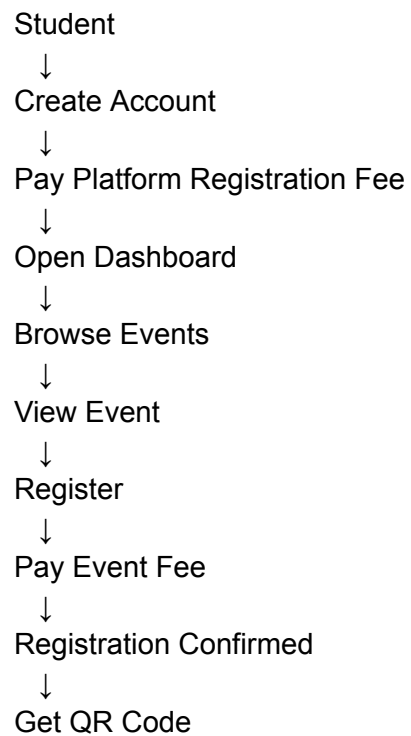
- View their assigned events
 - View registered students
 - Search participants
 - Scan student QR codes
 - Verify registration/payment
 - Mark attendance
 - View attendance
-

3. How does the platform work?

The overall flow is:



Then students can use the platform:



On the event day:

Student QR



Organizer Scans



System Checks Registration



System Checks Payment



Attendance Marked

4. Dynamic Website

This is one of the most important parts of the project.

We should **not hardcode event information into the frontend**.

For example, we should NOT build the website specifically around:

Tech Fest 2026

Coding Competition

₹500

500 Participants

Instead, the Admin enters this information through the dashboard.

For example:

Admin



Create Domain → Technology



Create Category → Coding



Create Event → Coding Competition



Set Fee → ₹500



Set Capacity → 500

The website automatically displays the information to students.

If next year the Admin creates completely different events, the same application should work.

5. Technology We Plan to Use

Frontend

- React
- TypeScript
- Bootstrap/Tailwind

Backend / Cloud

- Firebase Authentication
- Firebase Firestore
- Firebase Cloud Functions
- Firebase Storage
- Firebase Hosting

Payments

- Razorpay

Code Management

- GitHub
 - GitHub Organization
 - GitHub Projects
-

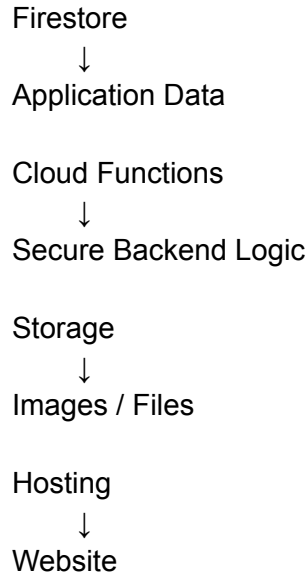
6. Why Firebase?

Firebase will provide most of the infrastructure we need.

Firebase Authentication



Login / Accounts



This allows our team to build the platform without maintaining a traditional server initially.

The expected scale of around **3,000–4,000 students** is reasonable for this architecture when the database queries and registration/attendance logic are designed properly.

7. Why Razorpay?

There are two types of payments.

First Payment

Student pays the **platform registration fee**.

Student → Razorpay → Payment Verified → Platform Access

Second Payment

Student pays the **individual event fee**.

Student → Event Registration → Razorpay → Payment Verified → Registration Confirmed

Payment verification will happen through the trusted backend/Cloud Functions.

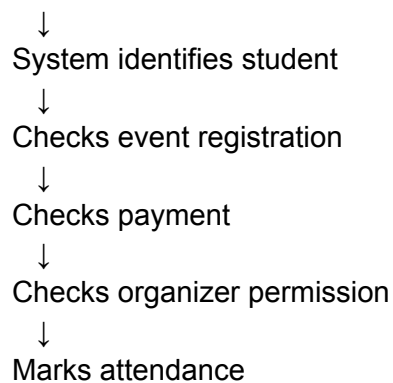
8. QR Code System

Each student gets a personal QR identity.

The QR code should not contain sensitive information directly.

When an organizer scans it:

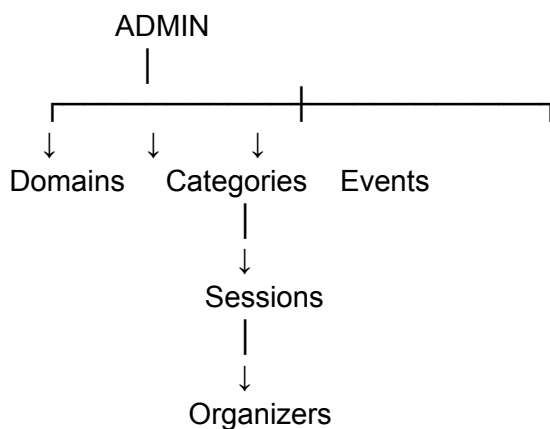
QR Scan

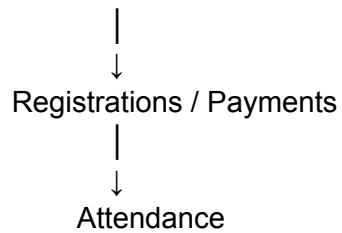


If the same student is scanned again, the system should avoid creating duplicate attendance.

9. Main Admin Dashboard

The Main Admin is basically the **control center** of the platform.





The Admin does not need developers to add a new event.

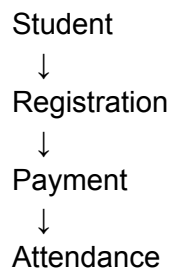
They should be able to configure it directly from the dashboard.

10. Basic Data Structure

The main relationship is:



And:



Organizers are assigned to specific events.

11. Important Rules

Rule 1

The platform starts **blank/configurable**.

Rule 2

The Main Admin creates the event structure.

Rule 3

Nothing event-specific should be hardcoded.

Rule 4

Students must complete platform registration before registering for events.

Rule 5

Interested and Registered are different.

Rule 6

Payments must be verified by the backend.

Rule 7

Organizers can only manage their assigned events.

Rule 8

QR codes are used for secure event verification.

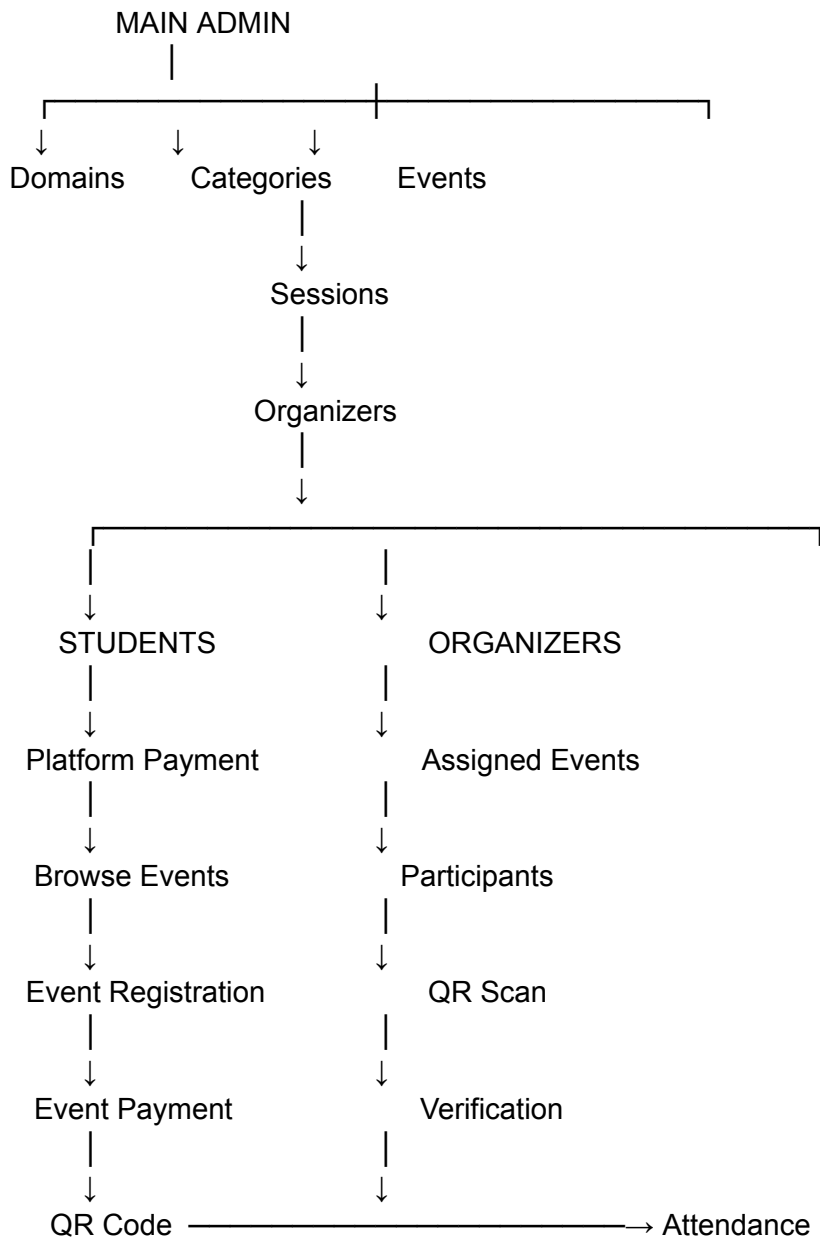
Rule 9

Attendance should not be duplicated.

Rule 10

Event capacity must be controlled by the backend.

12. Complete System in One Diagram



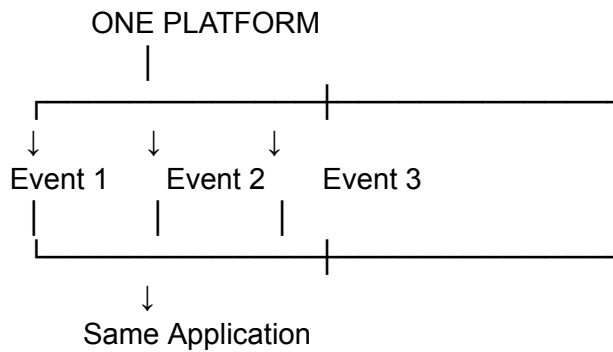
13. Final Idea

In simple terms:

We are building a reusable event platform where the Main Admin creates and controls the event structure, students register and pay through the platform, and organizers use QR codes to manage event attendance.

The application itself remains generic.

Only the **data and configuration change from event to event.**



This means we can use the same platform for this year's event and continue using it for future college events without rebuilding the website from scratch.